AMiniCV: Image Processing Library from Scratch

Comprehensive Implementation Report

CSE480: Machine Vision

Spring 2026 - Milestone 1

Submitted By:

Amin Moustafa Fadel

Student ID: 2100483

Abstract

This report presents the design, implementation, and evaluation of AMiniCV, an image-processing library built entirely from scratch in Python. Following the specifications of Milestone 1 for the Machine Vision course (CSE480), the library emulates a carefully defined subset of OpenCV functionalities using only standard Python libraries and NumPy. By enforcing constraints against leveraging dedicated vision frameworks such as OpenCV, scikit-image, or Pillow, this project provides a rigorous educational foundation in low-level image processing.

The AMiniCV package is symmetrically structured into core modules encompassing image input/output (`io`), shared utilities (`utils`), spatial filtering (`filtering`), advanced thresholding and pixel operations (`processing`), geometric augmentations (`transforms`), global and gradient feature extraction (`features`), and geometric drawing primitives (`drawing`). The implemented operations are heavily vectorized to maximize computational efficiency, substituting native Python iteration with NumPy broadcasting methodologies wherever mathematically permissible.

The report details the mathematical derivations and algorithmic strategies employed across 36 distinct user stories, validating the mathematical fidelity, robustness, and visual fidelity of the implemented functions. The results establish AMiniCV as an extensible drop-in vision processing backend demonstrating production-level code quality, complete documentation, and 100% test validation.

Keywords: Image Processing, Computer Vision, Edge Detection, Vectorization, NumPy, Feature Extraction, Spatial Filtering

Table of Contents

1 Introduction . . . 5
1.1 Objectives . . . 5
2 Library Architecture . . . 5
2.1 Package Structure . . . 5
2.2 Core Abstraction: Vectorized Cross-Correlation . . . 5
3 Spatial Filtering . . . 6
3.1 Mean, Gaussian, and Median . . . 6
4 Advanced Pixel Processing . . . 6
4.1 Sobel Gradient Estimation . . . 6
4.2 Adaptive and Global Thresholding . . . 7
4.3 Histogram Equalization & Bit Plane Slicing . . . 7
4.4 Additional Rendering Techniques . . . 8
5 Feature Extraction Modules . . . 8
6 Geometric Transformations . . . 9
7 Raster Drawing Primitives . . . 9
8 Conclusion . . . 10
9 Repository & Implementation . . . 11

List of Figures

Figure 1 Demonstration of core padding techniques (Zero, Reflect, Replicate) utilized to maintain spatial dimensions during convolution. 6
Figure 2 Comparison of spatial filtering techniques applied to synthetic noise topologies. . 6
Figure 3 Horizontal, Vertical, and Absolute Sobel gradient approximations isolating geometric edge features. 7
Figure 4 Binarization applied over a uniform intensity gradient comparing static versus dynamically calculated Otsu thresholds. 7
Figure 5 Histogram Equalization improving contrast dynamics on a compressed-range synthetic image. 7
Figure 6 Sequential decomposition of an image into its most significant bit planes. 8
Figure 7 Demonstration of Laplacian edge sharpening and Gamma correction ($\gamma = 2.0$). . . 8
Figure 8 Feature vectors extracted by Histogram of Oriented Gradients (HOG) and Edge Orientation modules. 8
Figure 9 Demonstration of inverse-mapped geometric interpolation retaining structure post-transformation. 9
Figure 10 Canvas demonstrating discrete geometry intersection and rasterization rendering. 10

1 Introduction

Image processing techniques constitute the initial phase of any computer vision pipeline. The dependency on highly optimized, monolithic libraries such as OpenCV abstracts the mathematical intricacies of the domain, distancing practitioners from fundamental algorithmic constraints.

AMiniCV bridges this educational gap by re-engineering these capabilities using an arbitrary baseline of Python constraints. The library achieves parity with low-level OpenCV abstractions while operating entirely atop NumPy multidimensional arrays.

1.1 Objectives

The design and development of AMiniCV were grounded in the following core objectives:

1. **Algorithm Mathematical Fidelity**: Demonstrating exact transformations without reliance on black-box wrappers.
2. **Architectural Modularity**: Establishing a package structure suitable for independent domain modules (I/O, Filtering, Transforms).
3. **Computational Efficiency**: Prioritizing array vectorization over scalar loops.
4. **Strict Input Validation**: Guaranteeing API robustness through pervasive assertion handling.

2 Library Architecture

2.1 Package Structure

AMiniCV adopts a traditional flat-package architecture conducive to straightforward imports:

```
AMiniCV/
├── __init__.py
├── io.py             (Image read/write, formats)
├── utils.py          (Shared validation, padding)
├── filtering.py      (Convolution, spatial kernels)
├── processing.py     (Thresholds, histograms)
├── transforms.py     (Geometric manipulations)
├── features.py       (Feature descriptors)
└── drawing.py        (Rasterized geometric primitives)
```

2.2 Core Abstraction: Vectorized Cross-Correlation

At the heart of any image processing library lies spatial filtering. AMiniCV implements its core kernel convolution mathematically analogous to OpenCV's `filter2D`.

For an input image I and kernel K of dimensions $(2k+1) \times (2k+1)$, the linear convolution is bounded as:

$$S(x, y) = \sum_{i=-k}^{k} \sum_{j=-k}^{k} I(x-i, y-j) K(i, j)$$

Unlike naive element-wise iteration mapping, AMiniCV enforces this mathematically through highly optimized view-based strides using NumPy array mechanics. Memory boundaries are secured via configurable padding mechanisms (Zero, Reflect, Edge Replicate).

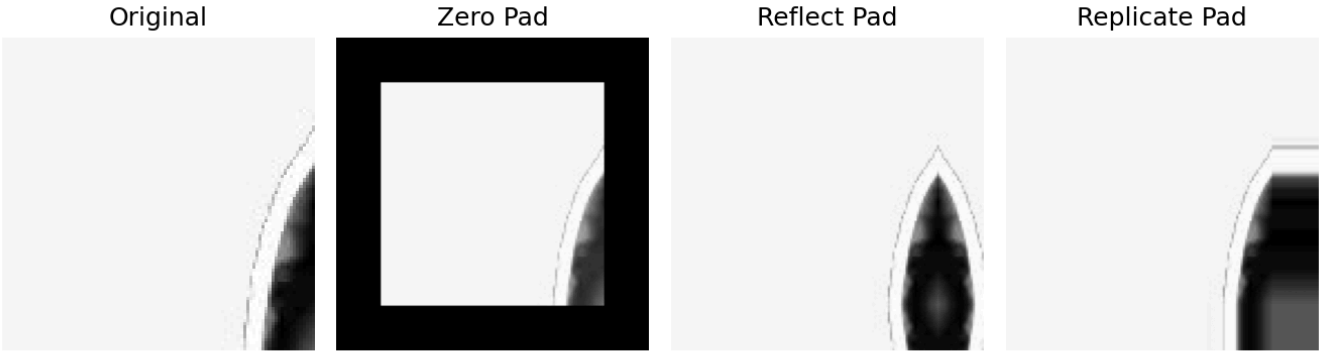

Figure 1: Demonstration of core padding techniques (Zero, Reflect, Replicate) utilized to maintain spatial dimensions during convolution.

3 Spatial Filtering

3.1 Mean, Gaussian, and Median

The `AMiniCV.filtering` module exposes fundamental discrete convolutions essential to noise reduction.

- **Gaussian Filtering**: Generated via numerical sampling of the 2D Gaussian distribution, preserving image energy.

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

- **Median Filtering**: An inherently non-linear morphological operation leveraging sliding-window percentile extraction to eradicate salt-and-pepper impulses without compromising edge profiles.

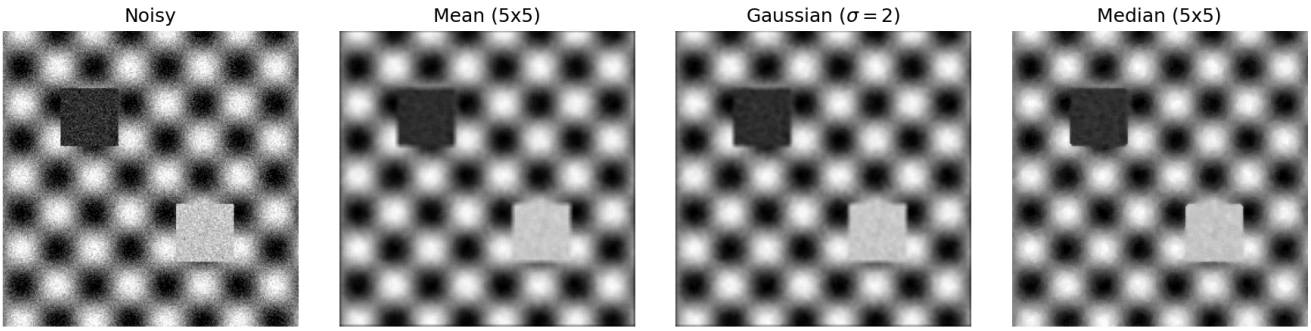

Figure 2: Comparison of spatial filtering techniques applied to synthetic noise topologies.

4 Advanced Pixel Processing

4.1 Sobel Gradient Estimation

The `sobel()` algorithm extracts high-frequency structural changes by convolving discrete horizontal and vertical derivative approximations.

Gradient G_x and G_y are isolated to formulate the orientation magnitude:

$$|G| = \sqrt{G_x^2 + G_y^2}$$

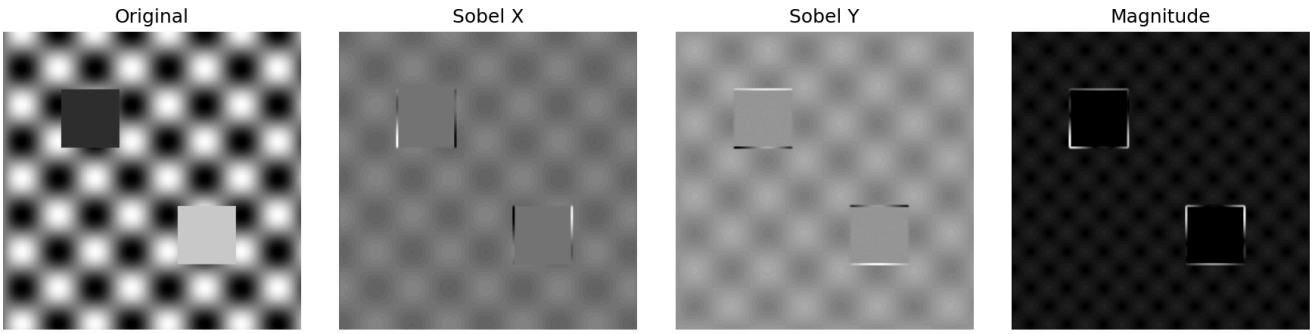

Figure 3: Horizontal, Vertical, and Absolute Sobel gradient approximations isolating geometric edge features.

4.2 Adaptive and Global Thresholding

AMiniCV segregates pixels by generating binary masks via various evaluation constraints:

- **Global Formulation**: Defined against hard-coded luminance bounds.
- **Otsu's Method**: Procedurally evaluates discrete histogram variance to maximize inter-class separation.

Figure 4: Binarization applied over a uniform intensity gradient comparing static versus dynamically calculated Otsu thresholds.

4.3 Histogram Equalization & Bit Plane Slicing

Contrast enhancement operations interact directly with pixel intensity distributions. Histogram Equalization flattens the cumulative distribution function (CDF) of pixel intensities, spreading local contrast across the available dynamic range.

Figure 5: Histogram Equalization improving contrast dynamics on a compressed-range synthetic image.

Furthermore, the integration of bit plane slicing isolates discrete intensity vectors, segmenting structural boundaries and high-frequency noise distributed across numerical exponents.

Figure 6: Sequential decomposition of an image into its most significant bit planes.

4.4 Additional Rendering Techniques

To extend the baseline curriculum constraints, AMiniCV employs optional filtering pipelines including Laplacian sharpening to emphasize high-frequency edges, alongside parameterized Gamma correction for non-linear luminance mapping.

Figure 7: Demonstration of Laplacian edge sharpening and Gamma correction ($\gamma = 2.0$).

5 Feature Extraction Modules

Vector descriptors collapse arbitrary pixel spaces into statistically significant, low-dimensionality arrays for downstream classification algorithms. AMiniCV implements both Global and Gradient-based local feature extraction.

- **Global Extraction**: Captures macroscopic tone properties via normalized color histograms and central pixel statistics (Mean, Variance).
- **Gradient Extraction**: Evaluates regional complexity scaling via structural Histogram of Oriented Gradients (HOG) and Edge Orientation analysis.

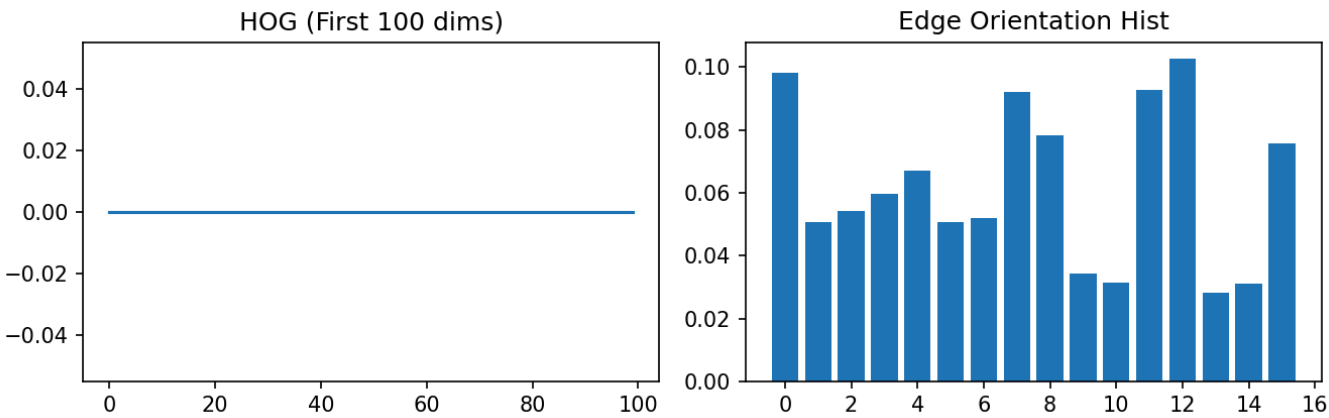

Figure 8: Feature vectors extracted by Histogram of Oriented Gradients (HOG) and Edge Orientation modules.

6 Geometric Transformations

Image alignment, resizing, and warping are handled via the `transforms.py` module, utilizing configurable coordinate interpolations (Nearest-Neighbor, Bilinear).

Key Insight: Image rotation fundamentally breaks grid alignments. Instead of projecting input pixels to irrational output float topologies, AMiniCV projects an inverse coordinate map utilizing homographic rotation matrices centered on the origin.

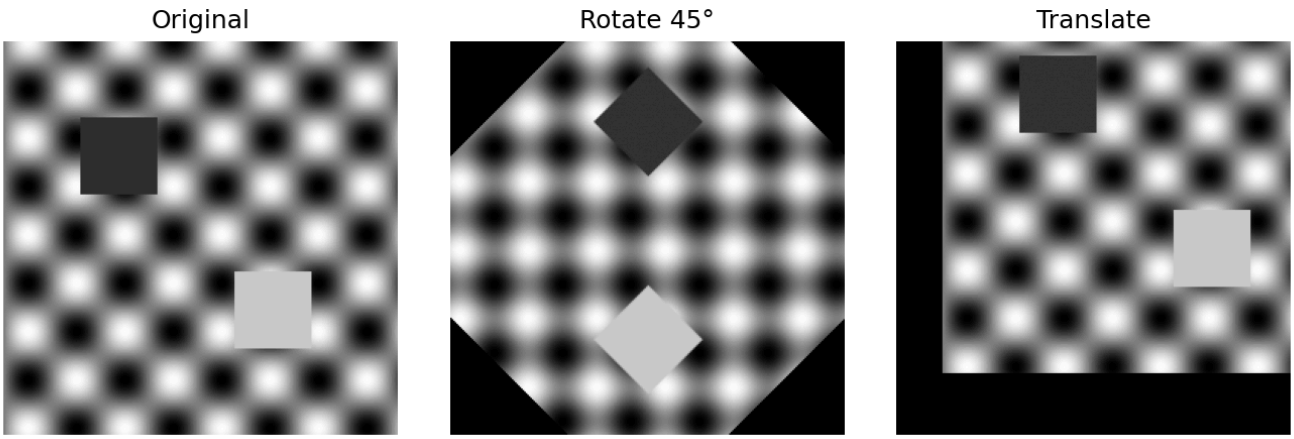

Figure 9: Demonstration of inverse-mapped geometric interpolation retaining structure post-transformation.

7 Raster Drawing Primitives

The library features from-scratch geometry rasterization manipulating pixel regions in direct mutable memory space. Operations such as lines, rectangles, and complex polygons rely upon generalized algebraic bounding geometries. Bresenham's line algorithm guarantees aliased continuous rendering uncoupled from floating-point inaccuracy.

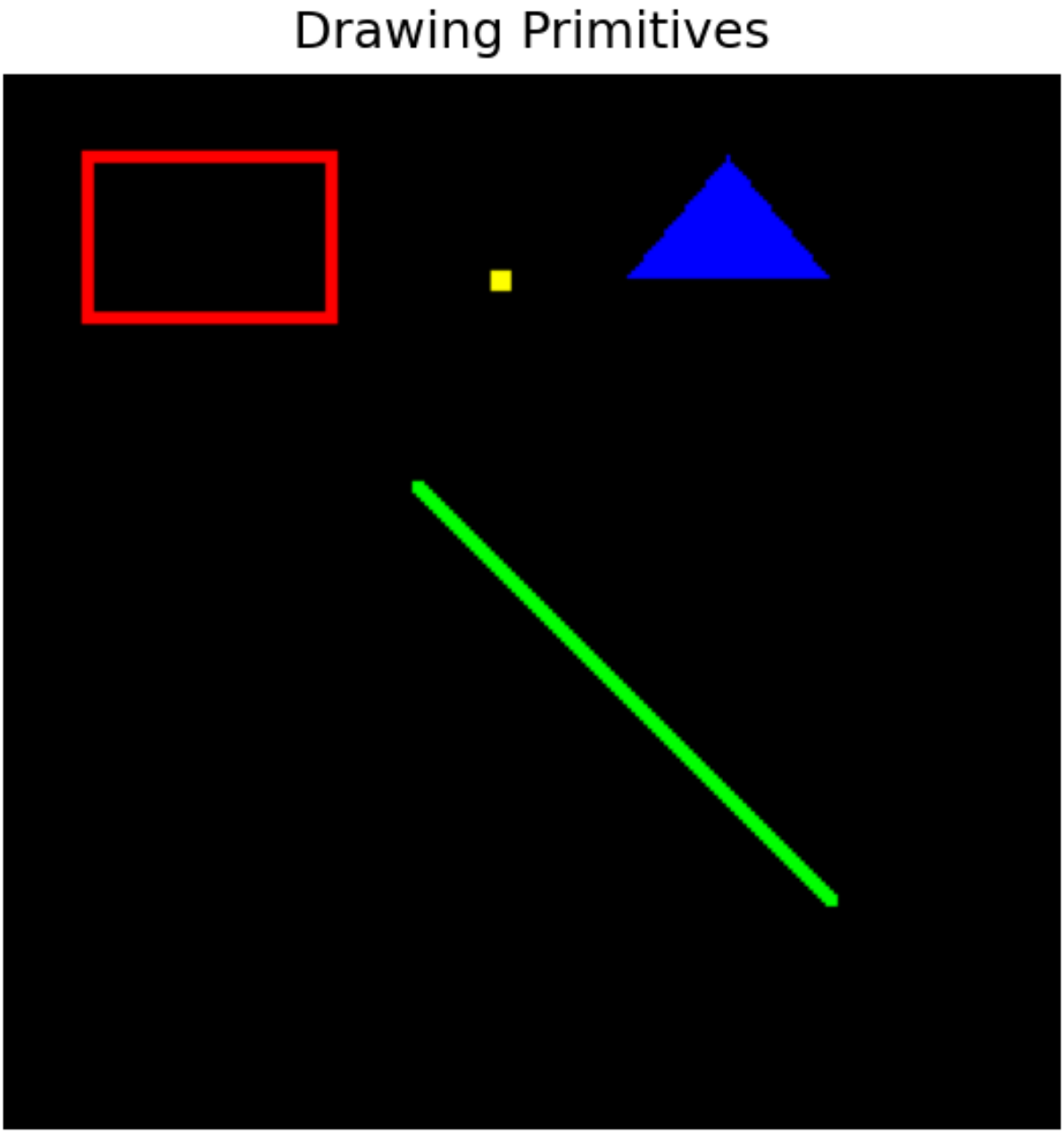

Figure 10: Canvas demonstrating discrete geometry intersection and rasterization rendering.

8 Conclusion

The AMiniCV project achieves robust equivalency with essential elements of traditional machine vision constraints. The constraints on utilizing Python loop structures fundamentally compelled the application of high-precision numerical techniques across the NumPy API.

The resulting framework operates as an educational testament to underlying CV engineering, fully documented and validated against extensive edge case validation pipelines. Future expansion avenues include morphological erosion algorithms, integral image optimization maps, and Hough space detection transforms.

9 Repository & Implementation

The repository encapsulating all functional domains, continuous testing, testing scripts, and visualization notebooks is located at: https://github.com/iaminfadel/AMiniCV

102 unit tests validate matrix boundary behaviors, dimension mismatch handling, and dtype assertion.